



CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

Sessions, Resume And Fork

Claude Code deep dive, episode 3

You continued yesterday's work and it went worse than starting over.

The obvious conclusion is that resuming is unreliable, so people stop doing it. That conclusion is wrong.

A session is a conversation tied to a project DIRECTORY, it starts empty unless you go and get the old one, and resuming a stale session is worse than starting a clean one.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- What a session is, and where it lives on disk
- What crosses from yesterday to today, and what does not
- The four ways to pick a session up
- Naming, and why six identical timestamps is the problem
- Resume against fork, which is the distinction that matters
- Git worktrees for genuinely parallel work
- Cross-session messaging

Out of scope

- Cloud sessions and Remote Control, which are episode 11
- Background agents and workflows, which are episode 8
- Scheduled work, which is episode 16
- Transcript formats and log parsing

A session has a useful lifespan. Three signals say it has ended, and they are in the checklist.

01 What crosses from yesterday to today

Conversation history	no
Files it had read	no
Command output	no
CLAUDE.md	YES, every session
Auto memory	YES, first 200 lines or 25KB
Everything else	gone unless you go and get it

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.



03 Tied to a directory, not a repository

That one word explains most of the confusing behaviour. Not your repo, not your account: a directory.

It is written to disk as you work, as a plaintext JSONL file, one line per message, tool use and result. So your history is a file you can read with ordinary tools.

And the reason a session does not appear where you expected it is almost always that you are standing somewhere else.

A session is a saved conversation tied to a project directory, stored locally as you work.

Claude Code docs, Manage sessions, checked September 2026

Each message, tool use and result is written to a plaintext JSONL file under `~/.claude/projects/`.

Claude Code docs, How Claude Code works, checked September 2026

04 Sessions are independent

Each new one starts with a fresh context window and none of the conversation history from previous sessions. Nothing carries over by default.

Two things cross the boundary deliberately: CLAUDE.md, read at the start of every session, and auto memory.

Everything else about yesterday is gone unless you go and get it.

Each new session starts with a fresh context window, without the conversation history from previous sessions.

Claude Code docs, How Claude Code works, checked September 2026

Claude can persist learnings across sessions using auto memory, and you can add persistent instructions in CLAUDE.md.

Claude Code docs, How Claude Code works, checked September 2026

05 Four ways in, and they are not interchangeable

`continue` picks up the most recent session in the directory you are standing in. Fast, and the one people use by reflex.

`resume` with a name reopens a specific one. `resume` with nothing opens a picker, showing sessions from the current worktree by default with shortcuts to widen the list.

And there is a flag to start from a pull request. Four doors into the same room, and the one you reach for without thinking is the least deliberate of them.

`claude --continue` finds the most recent session in the current directory.

Claude Code docs, Manage sessions, checked September 2026

The `/resume` picker shows sessions from the current worktree by default, with keyboard shortcuts to widen the list to other worktrees or projects.

Claude Code docs, How Claude Code works, checked September 2026

06 The distinction the episode turns on

`RESUME` reopens the session under the SAME identifier and appends your new messages to the existing conversation. One thread, getting longer.

`FORK` copies the history into a NEW identifier and leaves the original exactly as it was. Two threads, sharing a past.

Forking gives you what resuming cannot: a second approach that does not contaminate the first, and a known good conversation kept intact while you experiment beside it.

Resuming a session with `claude --continue` or `claude --resume` reopens it under the same session ID and appends new messages to the existing conversation.

Claude Code docs, How Claude Code works, checked September 2026

Forking with `--fork-session` or `/branch` copies the history into a new session ID, leaving the original unchanged.

Claude Code docs, How Claude Code works, checked September 2026

07 Why continuing yesterday made it worse

You continued a session that already had hours in it. The window did not start fresh; it started nearly full, and full of the wrong things: resolved debugging output, abandoned approaches, a file read once and never needed again.

So adding today's request triggered compaction immediately, and the decision it forgot was made early, in detail, which is exactly the profile of what goes.

You did not resume badly. You resumed a STALE session.

Detailed instructions from early in the conversation may be lost during compaction, while requests and key code snippets are preserved.

Claude Code docs, How Claude Code works, checked September 2026

A resumed session inherits the token cost of everything already in it, so it begins closer to the compaction threshold than a fresh one.

First principles, not vendor documentation

08 Real parallelism, because sessions are per directory

Two sessions in one directory are two conversations about the same files. Switching branches under a running session changes the files it sees while the conversation stays the same, which is how an agent ends up confused about which branch it is on.

Git worktrees give a separate directory per branch, and therefore a separate session per branch, with no crossover.

Once several are running, cross-session messaging lets one tell another when a long build finished, so the coordination you were doing by alt-tabbing happens between the sessions instead.

Since sessions are tied to directories, you can run parallel Claude sessions by using git worktrees, which create separate directories for individual branches.

Claude Code docs, How Claude Code works, checked September 2026

Claude sees the current branch's files, and when you switch branches the conversation history stays the same.

Claude Code docs, How Claude Code works, checked September 2026



09 The thirty second habit almost nobody has

Name your sessions. The rename command takes the name as an argument.

Without it, the picker is a list of timestamps you cannot tell apart: six sessions from Tuesday, same repo, same date, and a first line that is some variant of "look at the failing build".

Naming turns resuming from a guess into a choice.

Run `/rename` with a new name to name the current session, and resume it later by that name.

Claude Code docs, Manage sessions, checked September 2026

A picker full of identically-named sessions provides no information, so the cost of not naming them is paid every time you resume.

Operational practice, not vendor documentation

Three signals a session is over, and what to do

1. The task has changed: start fresh
2. The window is full of resolved work: start fresh
3. You are trying a second approach to one problem: FORK
4. Would this conversation help tomorrow, or just weigh on it?
5. Is the decision you need recorded in CLAUDE.md, or only here?
6. Are you in the directory you think you are in?
7. Do you need two branches at once? Use worktrees, not one session
8. Have you named it so you can find it in a week?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

Manage sessions

<https://code.claude.com/docs/en/sessions>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.